

Received 20 September 2024, accepted 16 October 2024, date of publication 22 October 2024, date of current version 30 October 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3484588

RESEARCH ARTICLE

Modeling and Evaluating a Cache System in ICN Routers Using a Programmable Switch and Computers

JUNJI TAKEMASA¹, (Member, IEEE), YUKI KOIZUMI¹, (Member, IEEE),
AND TORU HASEGAWA², (Member, IEEE)

¹Graduate School of Information Science and Technology, Osaka University, Osaka 565-0871, Japan

²Faculty of Materials for Energy, Shimane University, Matsue, Shimane 690-8504, Japan

Corresponding author: Junji Takemasa (j-takemasa@ist-osaka-u.ac.jp)

This work was supported by JSPS KAKENHI under Grant 23K28073 and Grant 22K14258.

ABSTRACT Information-centric networking (ICN) is one of promising networking architectures to replace IP because its notable feature, in-network caching, is expected to reduce about a one-third of the forever increasing Internet traffic. However, the existing ICN router implementations with caches cannot come with the rapid increase of terabit-scale network bandwidth. An implementation on hardware-based router platforms like programmable switches is promising, but due to their limited memory capacity, cache stores should be located at external devices like computers. This makes packets being sent from and to the switch to cache them at the computers, which increases the number of ports for connecting them called *external ports*. That is, the number of ports for connecting the switch and external networks is reduced, and as a result, the packet forwarding rate is degraded. We analytically and experimentally show that a naive implementation decreases both the packet forwarding rate and the number external ports to almost half in the condition that the cache hit ratio is about 30%. To solve this bottleneck, this paper proposes the two algorithms, the cache admission and lazy response ones, to reduce the number of packets sent between the switch and the computers. In order to validate them, we evaluate the forwarding rate improvement by developing an analytical model and a prototype ICN router implementation on a programmable switch with computers. The evaluation shows that the proposed algorithms almost double the forwarding rate compared to a naive implementation without the proposed algorithms under realistic request patterns to data packets. The paper's contributions are two-fold: realistic implementation and comprehensive performance evaluation. First, we implement a terabit-class ICN router with cache functionality on a Tofino switch which provides 3.2 terabit/s (Tbps) forwarding rate with two pipelines. The implemented router achieves 1.075 Tbps forwarding rate in the condition that just one pipeline of the Tofino switch is used and that the cache hit ratio is about 30%. As far as we know, this prototype implantation is one of the first full-fledged ICN routers on a Tofino switch. Second, the performance of the proposed router is both analytically and experimentally evaluated, and the both results show the similar packet forwarding rate under realistic request patterns following the Zipf distribution, which the Internet traffic is believed to follow. Finally, the paper is extended from the conference paper so that the proposed algorithms are evaluated both analytically and experimentally, whereas they are mainly evaluated by simulation in that paper.

INDEX TERMS Caching, information-centric networking, programmable switch.

The associate editor coordinating the review of this manuscript and approving it for publication was Songwen Pei¹.

I. INTRODUCTION

Information-centric networking (ICN), specifically, content-centric networking (CCN) [1] or named-data networking (NDN) [2], is a promising Internet architecture due to

name-based forwarding, in-network caching and so on. Its communication is a request/response style, which is different from IP's unicast communication. Specifically, a client called a source sends an interest packet to request a data piece, and a server called a producer replies a data packet containing the requested data piece. This style has various benefits over IP. First, such both way communication naturally provides end-to-end flow control at the network layer. Second, since the data piece is specified by its name, there is no need to resolve the name to an address. Third, data packets are stored at caches of intermediate routers (cached), and as a result, data packets are replied from the routers rather than the servers when interest packet hit data packets stored at the caches. This in-network caching reduces both traffic amount and response time.

Among the benefits, traffic reduction due to in-network caching is promising because the Internet traffic is ever increasing. It is well known that the cache hit ratio is about 30% for the traffic which follows the Zipf distribution [3]. However, the existing ICN router implementations cannot catch up with the rapid increase of terabit-scale network bandwidth. Many ICN routers are software-based and thus their forwarding rate is at most about 100 Gbps [4], [5], [6]. In contrast, the implementation on hardware-based router platforms like programmable switches [7] is promising and the authors achieve terabit-class packet forwarding rate [8]. However, due to programmable switches' limited memory capacity, as far as we know, no hardware-based ICN router provides in-network caching.

This paper addresses the implementation of terabit-class ICN router with a cache on a programmable switch. Hereafter, we call a programmable switch just as a switch. Since a cache is not placed at the switch, the cache naturally should be placed at external devices like computers with a large capacity memory device. Hereafter, we call such an external device as just a computer. This means that an ICN router should be a combination of a switch and computers, where the switch forwards interest/data packets and the computers store data packets.

However, naive implementations waste switches' ports, which results in low forwarding rate as described below. Obviously, switch ports are used to connect a switch and computers, and the number of ports used for communication to and from external networks is reduced. Hereafter, we call these ports as *computer ports* and *external ports*, respectively. That is, the maximum packet forwarding rate is constrained by the sum of the above available external ports' bandwidth. Since all the data packets are sent from the switch to the computers to store them at the caches, many ports should be used to connect the switch and the computers. We analytically and experimentally show that a naive implementation decreases both the packet forwarding rate and the number external ports to almost half in the condition that the cache hit ratio is about 30%. This observation concludes that the reduction of traffic between the switch and the computers is a key to high-speed forwarding.

In the conference paper [9], we proposed the solutions to reduce redundant traffic in the both directions between a switch and computers. The first solution is *cache admission*, which is adopted by the switch to avoid transferring unpopular data packets to computers. Such unpopular data packets are unlikely to incur cache hits later and therefore need not to be inserted into caches at computers. The second solution is *lazy response* of data packets upon cache hits based on the observation that a popular data packet is requested at $O(1\mu s)$ intervals, while the RTT on the Internet is $O(10ms)$ [10]. If we delay the responses by a few ms, only one data packet is transferred to the programmable switch.

The paper extends the conference paper, where the algorithms are mainly evaluated by simulation, in that they are extensively evaluated by the analytical model and the full-fledged prototype implementation. To validate that the ICN router with the proposed algorithms achieve terabit-class forwarding rates under realistic data piece request patterns, we develop the analytical model and the full-fledged ICN router with the two algorithms. Hereafter, we call this router as the proposed ICN router. The contributions of the paper are summarized below:

First, we develop the analytical model to calculate the average packet forwarding rates as well as cache hit rates and the reduction rates of the traffic between a switch and computers under static request patterns according to the Zipf distribution. The analysis results show that the proposed ICN router achieves higher packet forwarding rate than a naive ICN router without the proposed solutions. In addition, the analytical model is useful to decide the number of ports used to connect a switch and computers.

Second, we compare the packet forwarding rates of the prototype ICN router and those of the analytical model to validate that the analytical results are realistic under realistic request patterns. The comparison results clarify that the both rates are similar, which supports the correctness of the above evaluation results.

Third, we experimentally evaluate the packet forwarding rates with the prototype of the proposed ICN route under dynamic request patterns, where popularities of data pieces change in time. The evaluation results show that the packet forwarding rates do not degrade when they change.

Finally, the prototype router which is implemented on a Tofino switch achieves about 1.075 Tbps forwarding rate, while the naive implementation without the two algorithms just achieves 0.491 Tbps forwarding rate.

The rest of this paper is organized as follows: Section II develops the analytical model for estimating the maximum packet forwarding rates. Section III designs the two mechanisms for resolving the forwarding bottlenecks. Section IV develops the analytical model for estimating the cache hit ratio and the two reduction ratios in the both directions. Section V calculates the maximum packet forwarding rates under various traffic loads, and Section VI validates the applicability of the analysis results to realistic hardware using a prototype router implementation. Section VII

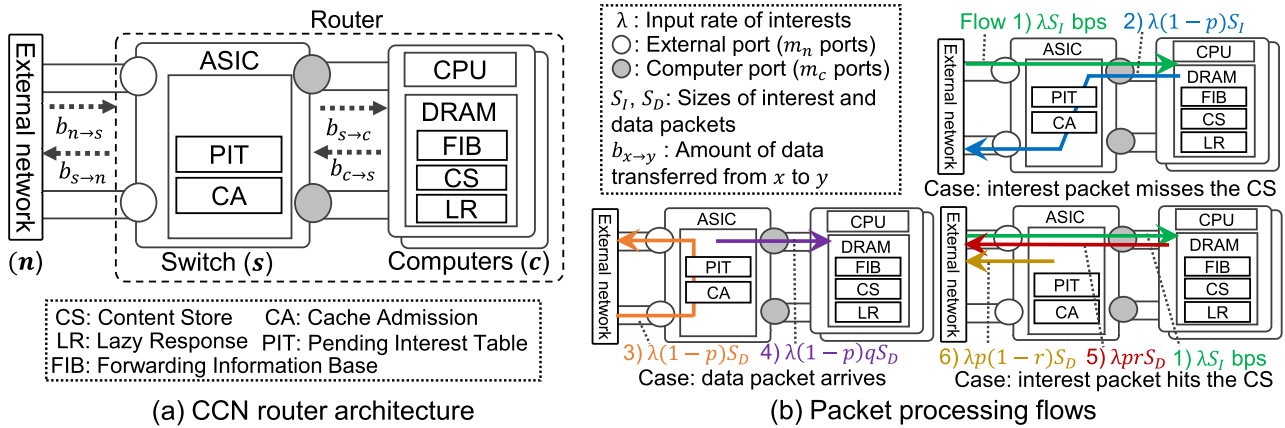


FIGURE 1. System architecture and packet processing flows of CCN router.

summarizes related work, and Section VIII concludes this paper.

II. CACHE ARCHITECTURE OF NAIVE AND PROPOSED CCN ROUTERS

In this section, we present a system architecture for a cache system consisting of a switch and computers. As discussed in Section I, deployment of the cache system on the Internet is promising for reducing ever-increasing Internet traffic. However, the cache system is not trivial to handle such large volume Internet traffic, i.e., terabit-class traffic, as most existing cache systems use computers as hardware platforms, which achieves at most 100 Gbps throughput. In contrast, combination of a switch and multiple computers is a reasonable hardware platform, as the latest switch can handle Tbps-class traffic [7].

The objective of this section is twofold: First, we define the hardware platform and the architecture of a cache system to clarify the performance challenges. This also provides guidelines for the design of high-speed cache systems in Section III. Second, we formulate an analytical model for the cache system to quantitatively prove these guidelines.

A. SYSTEM MODEL

We employ a switch-based CCN router [9] as the naive architecture because it achieves terabit-class throughput for both caching and name-based forwarding. In this subsection, we describe this naive architecture after briefly describing the CCN protocol and the router's hardware platform. For ease of reference, we summarize the technical terms and the symbols that we use in Tables 1 and 2, respectively.

1) CCN PROTOCOL

CCN provides a request-response communication of a named data piece between a client, called a source, and a server, called a producer. The source requests a data piece by sending an interest packet, which is a request packet, and the producer returns a data packet containing the corresponding data piece

TABLE 1. Terminology.

Term	Description
Name	A unique identifier of a data piece, used to match the request and the response packets in CCN request-response communication.
Interest packet	A request packet of a data piece. It has a name of a requesting data piece.
Data packet	A response packet containing a requested data piece. It has a name of a data piece.
Content Store (CS)	A cache that stores data packets to reply to interest packets.
Forwarding Information Base (FIB)	A forwarding table for interest packets, which records the mapping of name prefixes and next hop routers.
Pending Interest Table (PIT)	A forwarding table for data packets, which records reverse paths of interest packets.
Cache Admission (CA)	A function that decides whether to cache incoming data packets or not. It reduces traffic from the switch to computers.
Lazy Response (LR)	A function that aggregates multiple interest packets for the same data packet in a certain duration. It reduces traffic from computers to the switch.

as a response. A router between the source and the producer can cache data packets and respond to the interest packet if the interest packet matches a data packet in the cache. Both of the interest packet and the data packet contain a name of the data piece, which is used for matching between the interest and the data packet.

The CCN router delivers interest and data packets to the producer and the source, respectively, by using three data structures: a Content Store (CS), a Forwarding Information Base (FIB) and a Pending Interest Table (PIT). The CS is a cache that stores data packets. The FIB is a forwarding table for interest packets, which records the mapping of the name prefix (i.e., the first part of the name) and the next hop (i.e., the next router to forward), to forward interest packets. In contrast, the CCN router uses the PIT, which keeps reverse paths of interest packets, to forward data packets. We will detail how these data structures are used on the router platform in Section II-B1.

2) HARDWARE PLATFORM

Figure 1(a) illustrates the hardware platform of the CCN router architecture and the packet processing flows. For simplicity, we hereinafter refer to a programmable switch as a switch. The switch has multiple ports and a switch ASIC, which contains an SRAM device. The computers have a CPU and a DRAM device. The FIB and CS data structures are stored on the DRAM devices of the computers, and the PIT data structure is stored on the SRAM device of the switch.

The symbols specifying the hardware platform are summarized below:

- m, m_n, m_c : The number of ports, external ports and computer ports of the switch, respectively. We classify ports as *external* and *computer* ports. External ports are used to connect the switch and the external network, while computer ports are used to connect the switch to the computers. Please note that $m = m_n + m_c$.
- B (bps): The bandwidth of a single port. The sum of the external ports' bandwidth and the sum of the computer ports' one are $m_n B$ bps and $m_c B$ bps, respectively.
- $x \rightarrow y$: The direction from x to y from the view point of the switch, where x and y are any of s, n and c . s is the switch, n is the external network, and c is the set of computers. Please note that either x or y is s . Four directions are defined: $n \rightarrow s, s \rightarrow n, c \rightarrow s$ and $s \rightarrow c$.
- $b_{x \rightarrow y}$ (bps): The bandwidth from x to y . The bandwidth means the maximum amount of data that can be transferred in one second from x to y in one second. Four types of bandwidths are defined: $b_{n \rightarrow s}, b_{s \rightarrow n}, b_{c \rightarrow s}$, and $b_{s \rightarrow c}$.

3) CCN PROTOCOL IMPLEMENTATION OF THE BOTH ARCHITECTURES

a: NAIVE ARCHITECTURE

The CCN protocol is implemented according to the architecture we propose in another paper [8]. The FIB and CS are stored at the computers and the PIT is stored at the switch. The packet processing flows are described later in Section II-B1.

b: PROPOSED ARCHITECTURE

The architecture proposed in the conference paper [9] reduces traffic in the directions of $c \rightarrow s$ and $s \rightarrow c$. The cache admission (CA) algorithm and the lazy response (LR) algorithm are proposed, where the CA is implemented at the switch and the LR is implemented at the computers. The details are described later in Section III.

In the rest of this paper, we define the ratios of traffic reduced by the two mechanisms as follows:

- q : The ratio of data packets reduced in direction $s \rightarrow c$. The cache admission algorithm prevents data packets which are unlikely to hit later from being cached at the CS. Here, $q = 1$ means that the cache admission algorithm is not used in the naive architecture, while $q = 0$ means that the cache admission algorithm is used and completely reduces data packet traffic in direction $s \rightarrow c$.

- r : The ratio of data packets reduced in direction $c \rightarrow s$. The lazy response algorithm batches multiple responses to interest packets requesting the same data packet into one by delaying the responses (except for the last response). $r = 1$ means that the lazy response algorithm is not used in the naive architecture, while $r = 0$ means that the lazy response algorithm is used and completely reduces data packet traffic in direction $c \rightarrow s$.

4) PACKET TRANSFER TIME AND PACKET PROCESSING TIME

This paragraph identifies which of ports' bandwidth or packet processing time determines the maximum packet forwarding rate, assuming a modern programmable switch and modern CPU devices.

a: ASSUMPTION

The assumptions on the packet forwarding rate analysis of this paper are summarized below: First, data packet size S_D (1024×8 bits) is larger than interest packet size S_I (128×8 bits) because most applications follow request/response style communication like HTTP. Second, the following switch and the computer are assumed. The switch is the Intel Tofino 2 switch [7], which has 128 100 Gbps ports and an ASIC capable of processing 6×10^9 packets per second. The computer is the Supermicro server [11] equipped with two latest AMD CPUs [12], each with 64 2.0 GHz CPU cores and 128 PCIe 4.0 lanes, and four Intel E810 NICs [13], each providing two 100 Gbps ports using 16 PCIe 4.0 lanes.

b: BOTTLENECK TO FORWARDING RATE

The above assumption implies that ports' bandwidth is a factor of determining forwarding rate, not packet processing time. We compare the maximum packet input rates on external and computer ports and the maximum packet forwarding rates on the switch and the computer, respectively. First, we compare maximum packet input rate λ on external ports and the maximum packet forwarding rate of the switch's ASIC. λ is determined by the sum of the external ports, and is calculated as $12.8 \text{ Tbps} / (S_I + S_D \text{ bits}) = 1.38 \times 10^9$ packets/s. In contrast, the maximum packet forwarding (switching) rate of the ASIC is 6×10^9 packets/s, which is much faster than the maximum packet input rate determined by the bandwidth sum of the external ports. This concludes that the ASIC's maximum switching rate is not this factor.

Second, in a CCN software implementation [5], one CPU core can process 5×10^6 pairs of interest and data packets, and thus a computer with 128 CPU cores can process 6.4×10^8 pairs. However, 6.4×10^8 data packets consume $(6.4 \times 10^8) \times (1024 \times 8 \text{ bits}) = 41.94 \text{ Tbps}$ and the bandwidth is much larger than 100 Gbps of the computer port. The maximum data packet forwarding rate is not limited by the CPU's computation capability, but by the bandwidth of the computer ports.

TABLE 2. Summary of symbols.

Symbol	Description
λ (packets/s)	The maximum input rate of interest packets.
$\lambda_{\max}$ (packets/s)	The maximum packet forwarding rate.
m, m_n, m_c	The number of ports, external ports and computer ports of the switch, respectively.
B (bps)	The bandwidth of a single port.
$b_{x \rightarrow y}$ (bps)	The bandwidth from x to y .
$\lambda_{x \rightarrow y}$ (packets/s)	The packet forwarding rate constrained by bandwidth consumption in the direction from x to y .
S_I, S_D (bits)	The sizes of interest and data packets, respectively.
p	The cache hit ratio
q	The ratio of data packets reduced in direction $s \rightarrow c$ by the cache admission algorithm.
r	The ratio of data packets reduced in direction $c \rightarrow s$ by the lazy response algorithm.

B. ANALYTICAL MODEL

This sub-section develops an analytical model for estimating maximum packet forwarding rate $\lambda_{\max}$, which is the number of interest and data packet pairs forwarded in one second, under the condition that p, q, r, m_n and m_c are given. $\lambda_{\max}$ is the minimum among the maximum packet forwarding rates in the four directions, because the previous sub-section shows that ports' bandwidths are the determining factor for the maximum packet forwarding rate. Here, p represents the cache hit ratio.

1) PACKET PROCESSING FLOWS

In the first step, we identify the six types of flows (flows 1 to 6 in Figure 1(b)) in the three cases of interest and data packet handling, to calculate the average traffic amounts (bandwidths) in the four directions: $n \rightarrow s, s \rightarrow n, s \rightarrow c$ and $c \rightarrow s$. Here, the received packet rate from the external ports is assumed to be λ .

The first case is when a switch receives an interest packet, but the packet does not hit the cache on the computer. This interest packet is forwarded to the external port of the next hop. Here, flows 1 and 2 are defined.

- Flow 1: Interest packets are transferred from the external port to the computer port. The average bandwidth of this flow on the external and computer ports is λS_I bps.
- Flow 2: Interest packets that miss the cache are transferred from the computer port to the external port. The average bandwidth of this flow on the computer and external ports is $\lambda(1-p)S_I$ bps.

The second case is when the switch receives a data packet and sends back the data packet. In addition, the switch transfers this packet to the computer in order to cache this packet. Here, flows 3 and 4 are defined.

- Flow 3: Data packets are transferred from the external port to the switch, and then from the switch to the external port. The average bandwidth of this flow on the both external ports is $\lambda(1-p)S_D$ bps. Since interest packets that hit the cache are not forwarded to the next hop, the remaining data packets are responded. That is, $1-p$ data packets are responded.

- Flow 4: Data packets selected by the cache admission algorithm are transferred from the switch to the computer. The average bandwidth of this flow is $\lambda(1-p)qS_D$ bps on the computer port.

The third case is when the switch receives an interest packet, and the interest packet hits the cache on the computer. The switch sends this interest packet to the computer, and the computer responds the data packet. Please note that in Fig. 1(b), flow 1 described in this case is the same as flow 1 described in the case where the interest packet misses the CS. Here, flows 5 and 6 are defined.

- Flow 5: Response data packets that are not batched by the lazy response algorithm are transferred from the computer port to the external port. The average bandwidth of this flow is λprS_D bps on the computer and external ports.
- Flow 6: Response data packets that are batched by the lazy response algorithm are transferred from the switch to the computer port. The average bandwidth of this flow is $\lambda p(1-r)S_D$ bps on the computer port.

Please note that received interest packets are assumed to be sent equally to the computers using a load balancing mechanism [14]

2) MAXIMUM PACKET FORWARDING RATE

The sub-section calculates the bandwidths and the maximum packet forwarding rates in the four directions.

- $n \rightarrow s$: $b_{n \rightarrow s}$ is $\lambda(S_I + (1-p)S_D)$ bps. This rate is the bandwidth sum of flows 1 and 3. $b_{n \rightarrow s}$ is limited by the sum of the external ports' bandwidths, as shown by inequation $b_{n \rightarrow s} \leq m_n B$. That is, maximum packet forwarding rate $\lambda_{n \rightarrow s}$ in direction $n \rightarrow s$ is determined by the following equation:

$$\lambda_{n \rightarrow s} = m_n B / (S_I + (1-p)S_D) \quad (1)$$

- $s \rightarrow n$: $b_{s \rightarrow n}$ is $\lambda(S_I + (1-p)S_D)$ bps. This rate is the sum of the interest packet bandwidth of flow 2 and the data packet's bandwidth sum of flows 3, 5 and 6. $b_{s \rightarrow n}$ is limited by the sum of the external ports' bandwidths, as shown by inequation $b_{s \rightarrow n} \leq m_n B$. That is, maximum packet forwarding rate $\lambda_{s \rightarrow n}$ in direction $s \rightarrow n$ is determined by the following equation:

$$\lambda_{s \rightarrow n} = m_n B / ((1-p)S_I + S_D) \quad (2)$$

- $s \rightarrow c$: $b_{s \rightarrow c}$ is $\lambda(S_I + (1-p)qS_D) \leq (m-m_n)B$. This rate is the bandwidth sum of the interest packet bandwidth of flow 1 and the data packet bandwidth of flow 4. $b_{s \rightarrow c}$ is limited by the sum of the computer ports' bandwidths, as shown by inequation $b_{s \rightarrow c} \leq (m-m_n)B$. $\lambda_{s \rightarrow c}$ in direction $s \rightarrow c$ is determined by the following equation:

$$\lambda_{s \rightarrow c} = (m-m_n)B / (S_I + (1-p)qS_D) \quad (3)$$

- $c \rightarrow s$: $b_{c \rightarrow s}$ is $\lambda((1-p)S_I + prS_D) \leq (m-m_n)B$. This rate is the bandwidth sum of the interest packet bandwidth of flow 2 and the data packet bandwidth of

flow 5. $b_{c \rightarrow s}$ is limited by the sum of the computer ports' bandwidths, as shown by inequation $b_{c \rightarrow s} \leq (m - m_n)B$. $\lambda_{c \rightarrow s}$ in direction $c \rightarrow s$ is determined by the following equation:

$$\lambda_{c \rightarrow s} = (m - m_n)B / ((1 - p)S_I + prS_D) \quad (4)$$

As the result, $\lambda_{\max}$ becomes the minimum rate among the above four rates as shown by the following inequation.

$$\lambda_{\max} = \min(\lambda_{n \rightarrow s}, \lambda_{s \rightarrow n}, \lambda_{s \rightarrow c}, \lambda_{c \rightarrow s}) \quad (5)$$

III. DESIGN

As analyzed in Section II-A4b, wasting switch ports as computer ports is the bottleneck. For example, the analysis performed later in this section shows that about 40% of switch ports are wasted as computer ports, which degrades the throughput by 42% from the ideal throughput. These results suggest that a key driver for high throughput is to reduce the traffic of caching data packets on computers.

In this section, we first design the two algorithms to reduce caching traffic transferred from the switch to the computers, and vice versa: *cache admission* and *lazy response*. We then demonstrate quantitatively how the traffic reduction achieved by these algorithms improves the throughput of the cache system.

A. CACHE ADMISSION

We introduce cache admission to reduce redundant data packet transfers in the direction from the switch to the computer. The idea behind this introduction is that a single data packet needs to be forwarded *only once* for each name so that the data packet of the name is inserted into the CS of the computer. This eliminates transfers of data packets which will not be inserted into the CS.

We employ TinyLFU [15] as the cache admission algorithm due to its lightweight-ness. Here, where frequencies of individual requests to data packets, i.e., interest packets, are recorded is an issue. Since each frequency is recorded for each name, about tens of gigabytes [15] are required for storing the frequencies. To resolve the issue, we use the two data structures: the *frequency histogram* at the computer and the *insertion table* at the switch, as illustrated in Figure 2.

The frequency histogram is stored at the DRAM devices at the computer, and the TinyLFU algorithm is executed on the computer. When receiving an interest packet, the algorithm updates the frequency of the name. It decides the name of the next data packet that should be inserted to the CS, both if the frequency of the name becomes larger than the threshold, and if the data packet is not yet inserted into the CS. The threshold is determined as the frequency of a data packet randomly chosen from data packets stored in the CS. Then the algorithm at the computer lets the switch know the next name by setting the flag of the interest packet which is sent back to the switch.

The insertion table at the switch is used to maintain this one-bit flag for each name. When a data packet is received,

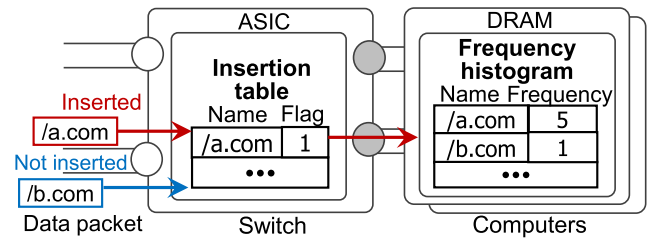


FIGURE 2. The design of the cache admission algorithm.

the switch checks the flag of the data packet's name. Only when the flag is 1, the switch forwards the data packet to the computer. The flag is reset to 0 so that data packets of the name are not forwarded, hereafter.

B. LAZY RESPONSE

1) DESIGN

We introduce a lazy response algorithm to reduce redundant data packet transfer in the direction from the computer to the switch. The idea behind the lazy response is to delay responses to interest packets requesting the same data packet in the CS so that only one data packet is transferred from the computer. Here, a response means a data packet as a reply to an interest packet.

The lazy response algorithm delays consecutive responses during the pre-defined threshold time and replies to a single response after the time expires. There are two design choices for checking the time expiration. The first and most naive choice is to use timers. However, it is unrealistic for high-speed routers to set a timer for each packet. Thus, we take the second choice, like *busy wait*. The computer checks whether the threshold time elapses or not every when an interest packet of a name in question is received. The details are described later in Section III-B3.

2) DATA STRUCTURE

Several fields are added to the lookup table of the CS for the lazy response, as illustrated in Fig. 3(a). Please note that each column records the information of the name in question. Field (P_i) is a flag indicating whether a response to port i is delayed according to the lazy response algorithm. Timestamp t_L is the time when the last interest packet is received. The initial value of t_L is T_M . Response time t_R is the time when the threshold time elapses after the first response is delayed.

3) PACKET PROCESSING FLOW

The interest packet processing flow is illustrated in Fig. 3(b), (c) and (d). Initially, the computer sends back data packets in the CS when receiving interest packets of a name in question. The first step, as illustrated in Fig. 3(b), is to decide to start delaying responses. The lazy response algorithm assumes that if the arrival interval of two consecutive interest packets is less than pre-defined threshold d_R , interest packets are received consecutively in a larger time than the pre-defined threshold time d_R . If the algorithm decides this start, it sends back the data packet and then sets $t_R = \text{the current time} + d_R$.

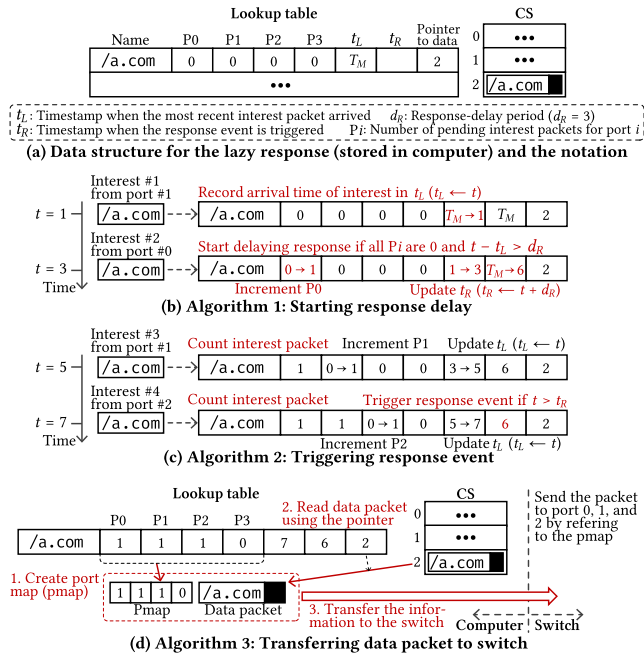


FIGURE 3. The Lazy response algorithm and the data structure.

The second step is to update port flags until time t_R , as illustrated in Fig. 3(c). Finally, the third and last step is to send back the data packet to the ports of which flags are set as 1 when the arrival time of the interest packet becomes later than t_R . Precisely, the computer adds field $pmap$, which is the list of the port flags, to let the switch know the port numbers to which the data packet is forwarded. After the last step, the lazy response algorithm re-starts from the initial step.

Since the lazy response algorithm does not use timers, there is a risk that a new interest packet is not received after time t_R . In this case, no data packet is replied for delayed responses. We introduce a background procedure like garbage collection to check such responses periodically.

C. MAXIMUM PACKET FORWARDING RATE ANALYSIS

The sub-section calculates the maximum packet forwarding rates ($\lambda_{\max}$ values) and analyzes which direction becomes the bottleneck for the rates: $s \rightarrow c$ or $c \rightarrow s$. Hereafter, we simply refer to the maximum packet forwarding rate as the maximum rate ($\lambda_{\max}$). For example, the fact that direction $s \rightarrow c$ is the bottleneck means that the data packet transfer from the switch to the computers determines the maximum rate.

In the first step, we calculate the maximum rates for the naive architecture, where neither algorithm is used, for various cache hit ratios (p values). This analysis identifies how many computer ports should be used and suggests how the maximum rate is improved by the two algorithms. In the second step, we calculate how the maximum rate ($\lambda_{\max}$) is improved by each algorithm in the condition that the cache hit ratio (p) takes a realistic value on the Internet, i.e., 0.3. The goal of this sub-section is to validate that the two algorithms are inevitably able to improve the maximum rate when cache hits reasonably occur.

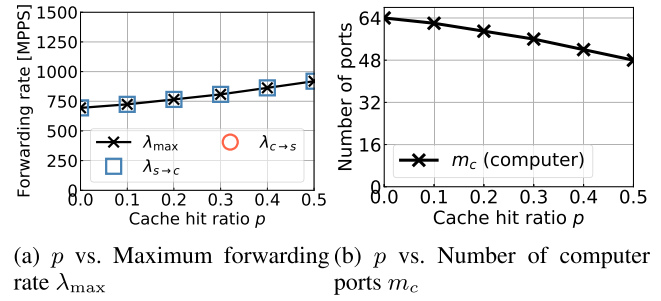


FIGURE 4. Performance of naive router under various values of cache hit ratio p .

1) NAIVE ARCHITECTURE

We calculate $\lambda_{\max}$ values and the number of computer ports (m_c) by varying cache hit ratios (p) between 0 and 0.5 with $q = 1$ and $r = 1$. Figure 4 (a) shows the maximum rates ($\lambda_{\max}$) for cache hit ratios. Here, each plot in the graph shows that the maximum rate is determined by either the $\lambda_{c \rightarrow s}$ or the $\lambda_{s \rightarrow c}$ value. This figure shows that the maximum rates are $\lambda_{c \rightarrow s}$ values for all the cache hit ratios. Figure 4(b) shows that the number of computer ports (m_c) value decreases as the cache hit ratio (p) increases.

The observation from the calculations is that direction $s \rightarrow c$ is the bottleneck that determines the maximum rate under the condition that the cache hit ratio (p) is between 0 and 0.5. At such realistic cache hit ratio values, data packets sent from the switch to the computers for caching consume many computer ports, thus limiting the maximum rate ($\lambda_{\max}$). When the cache hit ratio (p) is 0.4, the maximum rate is about 863 million packets per second (MPPS). This corresponds to 7.1 Tbps forwarding rate and this speed is just about 55% of the bandwidth sum of the external ports, i.e., 12.8 Tbps.

This observation leads to the first lesson:

The first lesson: The cache admission algorithm is expected to improve the maximum rate by reducing the number of computer ports because direction $s \rightarrow c$ is the bottleneck.

2) EFFECTS OF CACHE ADMISSION AND LAZY RESPONSE

a: CACHE ADMISSION

According to the first lesson, we calculate the maximum rates and the number of computer ports under the condition that the cache admission algorithm is used. We choose 0.3 as the cache hit ratio (p) and vary the q value between 1.0 and 0. $q = 1.0$ means that all the data packets are sent to the computers, and $q = 0$ means that none of data packets are sent to them due to the cache admission algorithm.

Figure 5 (a) and (b) show the maximum rate and the number of computer ports for q values. As q value decreases, the maximum rate increases and the number of computer ports (m_c) decreases. This shows that the cache admission algorithm increases the maximum rate ($\lambda_{\max}$). Here, an important observation is that the maximum rate increases when the q value becomes less than 0.4. When the q

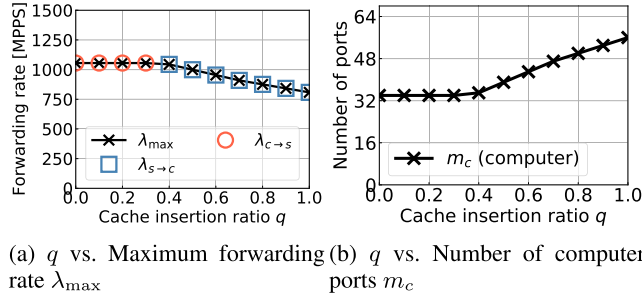


FIGURE 5. Performance improvement by cache admission (q changes. $p = 0.3$ and $r = 1.0$).

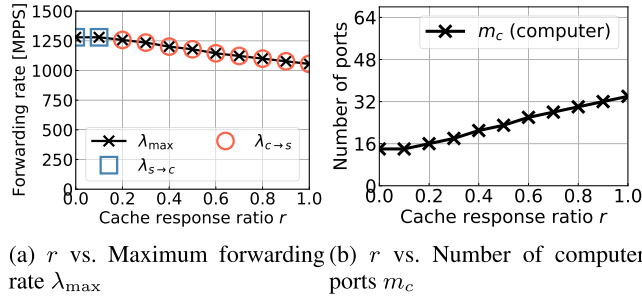


FIGURE 6. Performance improvement by lazy response (r changes. $p = 0.3$ and $q = 0$).

value is less than 0.4, direction $c \rightarrow s$ becomes the bottleneck and as a result, the $\lambda_{c \rightarrow s}$ value determines the maximum rate.

b: LAZY RESPONSE

The above result implies that traffic in direction $c \rightarrow s$ should be reduced when the cache admission algorithm reduces traffic in direction $s \rightarrow c$. We calculate the maximum rate and the number of computer ports to analyze how the lazy response algorithm improves the maximum rate when the cache admission algorithm reduces traffic in direction $s \rightarrow c$. Figure 6 (a) and (b) show the maximum rate and the number of computer ports for r values under the condition that $p = 0.3$ and $q = 0$. We choose 0 for q because Section V shows that q is close to 0 for realistic Internet traffic loads.

As the r value decreases, the maximum rate increases because the traffic in the direction $c \rightarrow s$ is reduced by the lazy response algorithm. Simultaneously, the number of computer ports (m_c) decreases. When the r value is between 0.2 and 1, direction $c \rightarrow s$ is still the bottleneck. However, when the r value becomes less than 0.1, direction $s \rightarrow c$ becomes the bottleneck again. Here, the important observation is that regardless of which direction is the bottleneck, the simultaneous use of both the algorithms is useful for improving the $\lambda_{\max}$ value. In the case where $p = 0.3$, $q = 0$, $r = 0.1$, the maximum rate becomes 1294 MPPS. This corresponds to 10.6 Tbps packet forwarding rate, which about 83% of 12.8 Tbps. Here, only $m_c = 14$ computer ports are used.

This observation leads to the second lesson:

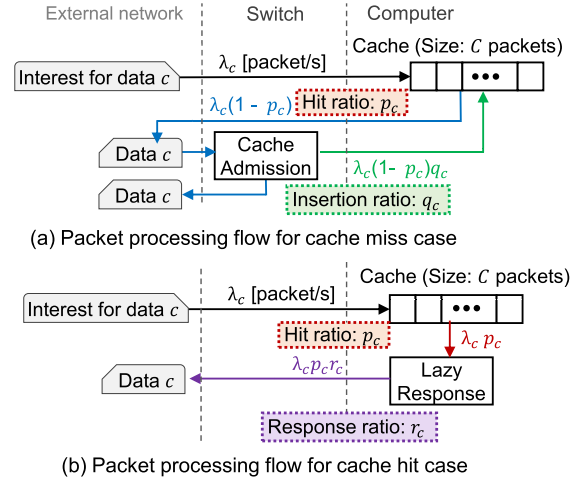


FIGURE 7. Assumed system model.

The second lesson: The lazy response algorithm is expected to improve the maximum rate when the cache admission algorithm sufficiently reduces the traffic in direction $s \rightarrow c$.

D. APPLICATION SCENARIO

The preliminary analysis in Section III-C clarifies that the simultaneous use of the cache admission and lazy response algorithms is useful to improve the maximum forwarding rate. However, the analysis assumes an ideal/optimistic scenario where p , q and r values are the same for all the data packets, i.e., their names. A remaining question is whether the analysis results of this section hold in realistic Internet environments where data packets' popularities follow some distribution like Zipf distribution. To answer this question, we analyze realistic p , q and r values for realistic data packet popularity distribution in the next two sections.

The analytical model plays a key role to deploy routers with the proposed algorithms in realistic Internet environments. An important decision in such a case is how many computers, i.e., computer ports, are connected to a switch. The optimal number should be selected to achieve the optimal packet forwarding rate. The analytical model, which will be developed in the next section, helps a network operator select the optimal number of computer ports for their assumed traffic characteristics.

IV. ANALYTICAL MODEL

In this section, we develop an analytical model of a cache system with cache admission and lazy response algorithms to derive the averages of cache hit, cache insertion and cache response ratios, p , q and r because they determine the forwarding bottleneck and the maximum forwarding rate, as discussed in Section III.

A. SYSTEM ASSUMPTIONS

We derive p , q and r by calculating averages for individual data packets. Let a data packet of interest be c ($1 \leq c \leq G$), where the number of data packets is G and they are numbered

from 1 to G without loss of generality. And let p_c , q_c and r_c be the cache hit, insertion and response ratios of packet c .

Figure 7 illustrates the system model which defines how incoming interest and data packets are handled by the cache system. The system consists of a cache (on a computer), a cache admission algorithm (on a switch) and a lazy response algorithm (on a computer). We assume multiple caches (deployed on multiple computers) to be *one big cache* to simplify the system model, which does not affect cache hit rate ratios [16]. The cache can store C data packets and employs RANDOM eviction algorithm. The cache admission algorithm decides whether to insert incoming data packets into the cache or not according to TinyLFU algorithm [15]. In the lazy response algorithm, an interest packet requesting data packet c that first hits the cache is delayed in period D [ms] for the response and subsequent interest packets of c arriving during this period are also delayed for the responses. Interest packets requesting data packet c ($1 \leq c \leq G$) arrive with arrival rate λ_c [packet/s], and a set of arrival rates for data packets follows some distribution such as Zipf distribution. Without loss of generality, data packets are assumed to be sorted in decreasing order of their arrival rate (i.e., $\lambda_c > \lambda_{c+1}$).

Our analysis assumes the static traffic scenario, where interest packets arrive according to Poisson process with static rate λ_c , while the realistic traffic is dynamic, wherein λ_c changes over time. The analysis is simple yet effective to understand the average performance in a macroscopic view; however, it cannot capture the temporal performance drops during transition periods, i.e., popularity changing. We evaluate such performance drops by using the implementation later in Section VI.

B. CACHE HIT RATIO

We model a cache hit ratio for data packet c (p_c) with an existing RANDOM cache model [17], which is based on a cache state transition with two states (*In* and *Out* states). States *In* and *Out* mean that data packet c is in the cache, and not in it, respectively. Figure 8 illustrates the cache state transition for data packet c . Data packet c , which is not stored at the cache, become stored at the cache if the frequency of the interest packet requesting this data packet c is larger than the threshold, that is, the cache admission algorithm admits it. Here, the arrival rate of such interest packets is q_c and the rate of admission is q_c . In contrast, data packet c , which is in the cache, is evicted from the cache after it has been in the cache for duration τ , which will be derived later.

p_c is derived as the probability that the cache state is *In* state. That is, *In* state means data packet c is in the cache and that interest packets of requesting data packet c hit the cache in this state. Such a probability is obtained by solving the equilibrium equation for a Markov chain with the two states, which is expressed as:

$$p_c = \frac{\lambda_c q_c \tau}{1 + \lambda_c q_c \tau}, \quad (6)$$

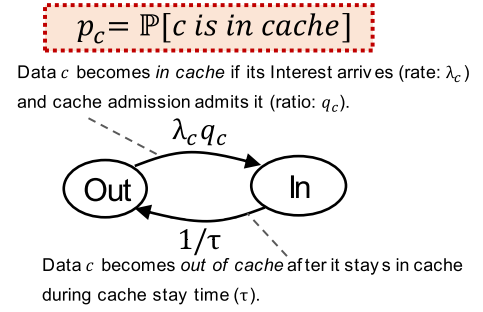


FIGURE 8. Modelling cache hit ratio for data packet c , p_c , based on a cache state transition of two states (which c is in and out-of cache).

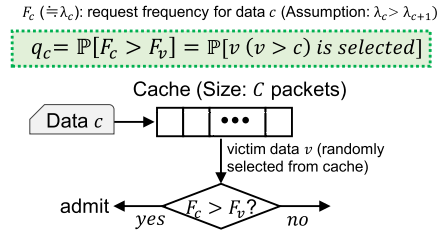


FIGURE 9. Modelling cache insertion ratio for data packet c , q_c , based on TinyLFU admission behavior.

We can obtain τ for a cache of size C by solving equilibrium equation $C = \sum_{k=1}^G p_k$ [17]. Finally, cache hit ratio p is the average of p_k for all data packets ($1 \leq k \leq G$) such as:

$$p = \frac{\sum_{k=1}^G \lambda_k p_k}{\sum_{k=1}^G \lambda_k}. \quad (7)$$

C. CACHE INSERTION RATIO

We derive a cache insertion ratio for data packet c by rigorously reproducing TinyLFU's admission behavior, shown in Figure 9. When data packet c arrives, TinyLFU randomly selects a data packet from the cache, which is called a *victim* data packet, and allows data packet c to be cached if its request frequency of data packet c (F_c) is larger than that of victim data packet v (F_v). That is, q_c is derived as a probability that v is a data packet satisfying condition $F_c > F_v$, which is expressed as $q_c = P[F_c > F_v]$.

Here, request frequency F_c is the number of interest packets that arrive during a certain period. This means that F_c is regarded as the arrival rate of interest packets for data packet c , λ_c , because λ_c is constant regardless of time. Hence, condition $F_c > F_v$ is equal to $\lambda_c > \lambda_v$. Since $\lambda_c > \lambda_{c+1}$ is assumed in Section IV-A, $P[F_c > F_v]$ is rewritten as $P[F_c > F_v] = P[\lambda_c > \lambda_v] = \sum_{v=c+1}^G P[X = v]$, wherein the victim data packet is data packet v .

$P[X = v]$ is derived as the joint probability of the probability that data packet v is in the cache (i.e., p_v) and the probability that data packet v is randomly chosen from the cache whose size is C data packets (i.e., $1/C$). In summary, q_c is expressed as:

$$q_c = \sum_{v=c+1}^G p_v (1/C). \quad (8)$$

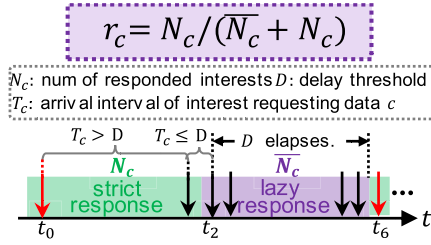


FIGURE 10. Modelling cache response ratio for data packet c (r_c) based on a time sequence of interest packet arrivals.

Finally, cache insertion ratio q is the average of q_k of all data packets ($1 \leq k \leq G$) such as:

$$q = \frac{\sum_{k=1}^G \lambda_k (1 - p_k) q_k}{\sum_{k=1}^G \lambda_k (1 - p_k)}. \quad (9)$$

D. CACHE RESPONSE RATIO

We derive a cache response ratio for data packet c based on a time sequence of interest packets of requesting this data packet, shown in Figure 10. In this figure, we assume that data packet c is in the cache because lazy response algorithm is a technique for interest packets that hit the cache. In the sequence of interest packet arrivals, two phases alternately occur: *strict response phase*, where the interest packet is responded to immediately, and *lazy response phase*, where the interest packet is responded to in conjunction with interest packets later arriving (i.e., not responded to at the arrival time). Let N_c and $\bar{N}_c$ be the number of interest packets during the strict response phase and that during the lazy response phase, respectively. r_c is the ratio of the number of interest packets arriving during the strict response (N_c) to the total number of interest packets ($N_c + \bar{N}_c$), which is expressed as $r_c = N_c / (N_c + \bar{N}_c)$.

We derive N_c by exploiting the following observation in Figure 10: The strict response phase switches to the lazy response phase, when some interest packet arrives following the duration shorter than D . This means that N_c follows a geometric distribution with success probability that an arrival interval of two consecutive interest packets for c is shorter than D . Here, such a probability is derived from a cumulative distribution function of the Poisson process's arrival interval ($1 - e^{-\lambda_c D}$ for c) because interest packets for data packet c arrive according to the Poisson process with rate λ_c . Hence, N_c is derived as the expected value of this geometric distribution, which is expressed as $\mathbb{E}[N_c] = 1 + \sum_{k=0}^{\infty} k \mathbb{P}[N_c = k] = 1 / (1 - e^{-\lambda_c D})$.

In contrast, $\bar{N}_c$ is simply derived as $\mathbb{E}[\bar{N}_c] = \lambda_c D$ because the lazy response phase continues during period D and interest packets for data packet c arrives with rate λ_c , as described in Figure 10.

In summary, r_c is expressed as:

$$r_c = \frac{1}{1 + \lambda_c D (1 - e^{-\lambda_c D})}. \quad (10)$$

Finally, cache response ratio r is the average of r_k of hit interest packets for all data packets ($1 \leq k \leq G$) such as:

$$r = \frac{\sum_{k=1}^G \lambda_k p_k r_k}{\sum_{k=1}^G \lambda_k p_k}. \quad (11)$$

V. ANALYSIS RESULTS

In this section, we analyze what values p , q and r take for realistic Internet traffic loads as well as how the obtained values of p , q and r decide the maximum packet forwarding rate and the optimal computer port number. The analysis results clarify that the analytical model enables to calculate p , q and r values under static request patterns. We obtain the key finding from the comprehensive analyses.

- The cache size decides the cache hit, insertion and response ratios under the condition that the data packet popularity distribution is known and that they are statically requested. As a result, we can decide the optimal number of computer ports when the number of switch's ports and cache sizes of computers are given, by using the above three ratios. That is, the analytical model is useful to choose the optimal cache size and the computer port number to achieve the maximum forwarding rate, if the data packet popularity distribution is known. (Sec. V-B)
- The lazy response algorithm significantly reduces data packet traffic in direction c to s , when the popularity skewness in the traffic load is high. In contrast, it does not reduce so much for low skewness traffic load. However, this small reduction is not problematic because the cache hit ratio is small under such traffic load, meaning that there is almost no data packet traffic in direction c to s regardless of the use of the lazy response algorithm. (Sec. V-C)
- The combined use of the cache admission and the lazy response algorithms achieves near maximum packet forwarding rate by almost completely reducing data packet traffic in both directions. Both of the algorithms are necessary for high packet forwarding rate. (Sec. V-C)

A. ANALYSIS CONDITIONS

We use content object popularities based on CDN traffic to calculate p , q and r values because the CDN traffic accounts for a large portion of Internet traffic. We assume that 10^8 content objects are requested according to a Zipf popularity distribution with exponent 0.9 [3]. Each content object consists of 10^3 data packets, and each data packet composing the same object is requested by an interest packet according to a uniform popularity distribution. That is, the data packets of the same object have the same popularity. The delay threshold in the lazy response algorithm is set to 10 ms, which is much smaller than the RTT in the Internet (60 ms on average [10]). The sizes of interest and data packets, S_I and S_D , are 128×8 bits and 1024×8 bits, respectively.

The router configurations are summarized below: The cache size is 10^8 packets, which correspond to 100 Gbytes.

The switch composing the router has 128 100 Gbps ports, and thus the interest packet arrives at a router with rate 1.6×10^8 packet/s, which is calculated as $128 \times 100 \times 10^9 / S_D$. Other settings are the same as in Section II.

B. CACHE SIZE VERSUS FORWARDING RATE

Cache hit ratio p is determined according to a traffic load and a cache size, and thus, we calculate p , q and r values according to the analytical model in Section IV and then calculate $\lambda_{\max}$ and m_c values and according to the analytical model in Section II.

Figures 11(a), (b) and (c) show the maximum rate $\lambda_{\max}$ and computer port number m_c values, as well as p , q and r values under the condition that the cache size is between 10^7 packets and 10^9 packets, respectively. In the figure, emProposal means a router with cache admission and lazy response algorithms (the proposal router), while Naive means that without the two algorithms (the naive router).

1) COMPARISON (PROPOSAL VERSUS NAIVE)

In Fig. 11(a), the proposal router always achieves higher packet forwarding rate (the maximum rate $\lambda_{\max}$) than the naive router. Obviously, the computer port number m_c values of the proposal router are smaller than those of the naive router as shown in Fig. 11(b). Figure 11(c) shows that q and r values are always less than 1 for all the cache sizes. This means that the both cache admission and lazy response algorithms reduce redundant data packet transfers in both directions $s \rightarrow c$ and $c \rightarrow s$.

2) PROPOSAL ROUTER'S CHARACTERISTICS

When the cache size is small (e.g., 10^7 packets), the proposal router achieves 1251 MPPS, which is the near-ideal value of the forwarding rate as shown in Section III-C. Since cache hit ratios p is low, the data transfer in direction $c \rightarrow s$ is small. As a result, direction $s \rightarrow c$ is the bottleneck, and thus, the cache admission algorithm contributes to the improvement of the maximum rate $\lambda_{\max}$. Here, an important observation is that the cache admission algorithm almost completely reduces redundant data packet transfer in direction $s \rightarrow c$ as shown by the fact that q is always close to zero.

We also find that the the proposal router's forwarding rate decreases as the cache size increases. This decrease is contrary to the expectation that the high cache hit ratio achieves the high forwarding rate. The reason for this decrease is explained as follows: As cache hit ratio p increases, the data packet traffic in direction $c \rightarrow s$ increases and this direction becomes the bottleneck. In this case, the lazy response algorithm reduces the data traffic in direction $c \rightarrow s$. However, the lazy response algorithm cannot completely eliminate redundant data packet transfers. That is, r cannot be close to zero, as shown in Fig. 11(c). Even in the case where the cache size is 10^7 packets, r is about 0.3. This means that about 70% of data packets are transferred in direction $c \rightarrow s$.

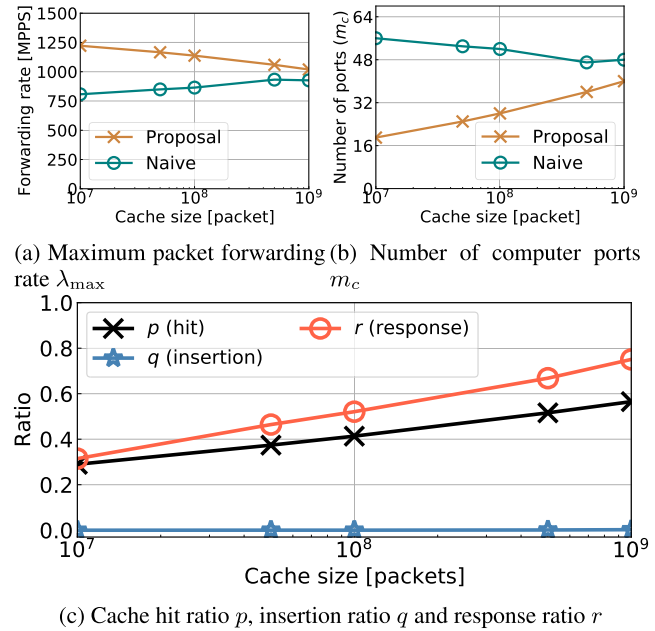


FIGURE 11. Cache size vs. performance.

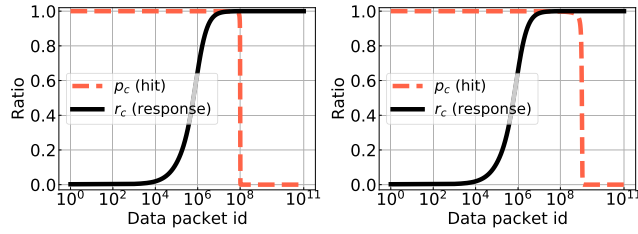
3) PACKET-LEVEL ANALYSIS OF LAZY RESPONSE

To investigate the reason why cache response ratio r increases as cache hit ratio p increases, we analyze the p and r values of individual data packets. Figure 12(a) and (b) show the p and r values of individual data packets under the condition that the cache size is either 10^8 or 10^9 packets. All the data packets are listed (ranked) in order of popularity and given unique identifiers in that order. Here, the smallest (highest ranked) identifier means that its popularity is the highest.

These figures show that there are three groups of data packets according to their popularities.

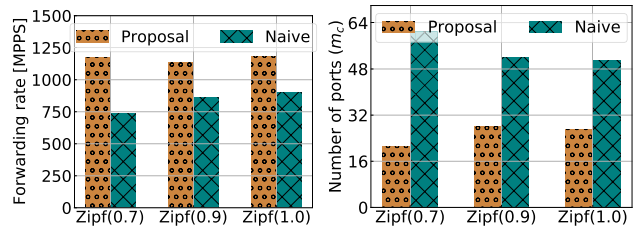
- High popularity: Interest packets with high popularities always hit the cache ($p = 1$), and the lazy response algorithm eliminates most data packets from the cache ($r = 0$).
- Middle popularity: Interest packets with middle popularities always hit the cache ($p = 1$), but the lazy response algorithm does not eliminate many data packets from the cache ($r > 0$).
- Low popularity: Interest packets with low popularities rarely hit the cache (p is almost 0), but the lazy response algorithm does not eliminate many data packets from the cache ($r = 1$).

Interest packets of the first and third groups do not pose any problem for high forwarding rates because data packets are rarely sent from the cache in direction $c \rightarrow s$. In contrast, interest packets of the second group increase the data packet traffic from the cache even if they always hit the cache. Here, an important observation is that the number of the second group packets is larger when the cache size is 10^9 packets than when it is 10^8 packets. This is the reason why the maximum forwarding rate decreases as the cache size increases.

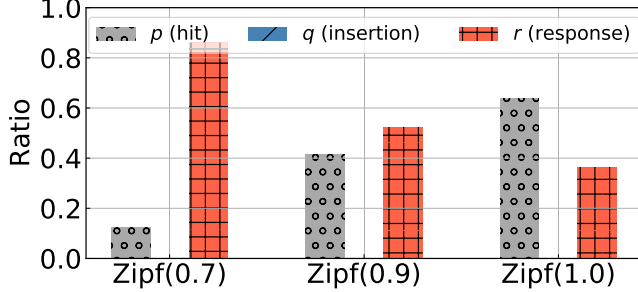


(a) when cache size is 10^8 packets (b) when cache size is 10^9 packets

FIGURE 12. Cache hit ratio ρ_c and cache response ratio r_c for each data packet c .



(a) Maximum packet forwarding rate $\lambda_{\max}$ (b) Number of computer ports m_c



(c) Cache hit ratio p , insertion ratio q and response ratio r

FIGURE 13. Skewness vs. performance (cache size is 10^8 packets).

C. POPULARITY SKEWNESS VERSUS FORWARDING RATE

The goal of this sub-section is to validate that the cache admission and lazy response algorithms improve the maximum forwarding rate for content object popularity distributions with different configurations of skewness. This sub-section uses different exponents for Zipf popularity distributions, 0.7 and 1.0, while the previous sub-section uses 0.9 as the exponent, where the exponents represent skewness of popularities. We refer to Zipf distributions with exponents 0.7, 0.9 and 1.0 as Zipf(0.7), Zipf(0.9) and Zipf(1.0), respectively.

1) COMPARISON (PROPOSAL VERSUS NAIVE)

Figure 13 shows the maximum packet forwarding rate $\lambda_{\max}$ values of the proposal and the naive routers for traffic loads of Zipf(0.7), Zipf(0.9) and Zipf(1.0). An important observation is that the proposal router consistently achieves high packet forwarding rates in all traffic load settings (1177 MPPS, 1137 MPPS, and 1179 MPPS in Zipf(0.7), Zipf(0.9), and Zipf(1.0), respectively), which are close to the ideal value

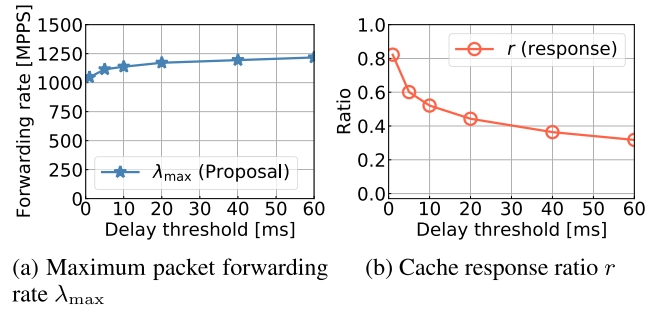


FIGURE 14. Delay threshold vs. performance of lazy response.

of $\lambda_{\max}$ in Section III-C. The numbers of external ports ($m_n = m - m_c$) are 107, 100, and 101, which are similar to each other like $\lambda_{\max}$ values.

2) PROPOSAL ROUTER'S CHARACTERISTICS

To investigate the reason why the proposal router consistently achieves near-ideal packet forwarding rate, we further analyze the cache hit ratio p , cache insertion ratio q , and cache response ratio r values, focusing on skewness of traffic loads. Figure 13(c) shows the p , q and r values for Zipf(0.7), Zipf(0.9), and Zipf(1.0).

The skewness has both positive and negative effects on the maximum forwarding rate. On the negative side, cache hit ratio p increases as the skewness increases. The traffic amount of data traffic in direction $c \rightarrow s$ increases. On the positive side, cache response ratio, r , decreases as the skewness increases, as shown in Fig. 13(c). These positive and negative effects cancel out each other under the condition that the skewness is between 0.7 and 1.0.

The observation is that the cache hit ratio and the data packet reduction ratio in direction $c \rightarrow s$ are balanced for realistic Internet loads, where the skewness is between 0.7 and 1.0 for Zipf data popularity distributions.

D. IMPACT OF DELAY THRESHOLD IN LAZY RESPONSE

We show the effect of delay threshold d_R on the performance of the lazy response algorithm because so far we assume that d_R is always 10 ms. Please recall the description in Section III-B, where d_R means that when the cache hit of the first interest packet occurs, the lazy response algorithm delays the response to this interest packet in d_R ms. We calculate r by changing d_R from 1 ms to 60 ms. Figures 14(a) and 14(b) show the maximum forwarding rate $\lambda_{\max}$ and cache response ratio r as d_R varies from 1 ms to 60 ms. r is minimal for the largest d_R , 60 ms; however, the increase in $\lambda_{\max}$ is not significant even as d_R increases. Since the larger delay such as 60 ms cancels out the RTT reduction due to cache hit, $d_R = 10$ ms is a good trade-off between data transfer reduction and RTT increase in our analysis conditions.

E. DISCUSSIONS

This subsection summarizes the implications obtained from the analysis results.

1) FORWARDING RATE LIMITATION FOR SMALL DATA PACKETS

The cache admission and lazy response algorithms do not improve forwarding rate for small data packets. This is because, if the average data packet sizes is as small as that interest ones, the reduction of the number of data packets does not contribute to the reduction of the traffic amount. However, such small data packets are not common in many real-world scenarios such as Internet traffic [18] of which average packet size is about 536 byte long.

In contrast, if we should assume small data packet sizes, techniques that reduce interest packet traffic at the switch (i.e., before the computer) are required. In-switch frontend caching [19] is a candidate technique.

2) APPLICABILITY TO OTHER USE CASES

The idea behind the two proposed algorithms, cache traffic reduction, is applicable to other use cases than this ICN cache case. They are useful to request-response style applications with large response data (e.g., the ratio of request to response size is 10 in our ICN cache case). One of the closest use cases is the CDN cache, where a http response (1KB to 1MB response [3]) is large relative to a http request (a few hundred bytes request). Another example is a Key-Value Store (KVS) cache where a value is requested by a key. Some recent production trace implies a similar request/response size ratio, such that for a key-value pair, the returned value size is about ten times larger than the requesting key size [20].

VI. EXPERIMENTAL EVALUATION

This section implements a router prototype and evaluates its performance in terms of two issues, which are not addressed by the model-based analysis in Section IV. The objectives are twofold: the first one is to validate that the proposed algorithms achieve forwarding rate similar to the analysis result in Section IV. The second objective is to evaluate the stability of the forwarding rate under time-varying traffic loads, which is difficult to evaluate with the analytical model.

A. PROTOTYPE IMPLEMENTATION

a: ROUTER PLATFORM

We implement the router prototype by using an Intel Tofino switch and two computers. The switch has two pipelines, each of which contains 16 100 Gbps ports [21]. We use one pipeline for a router prototype and the other for a tester, which generates test traffic for the router's pipeline. Each computer contains two 22-core Xeon CPUs [22] and two 100 Gbps NICs [13]. From the analytical model in Section II, the optimal number of computer ports is derived as four. Thus, the router's pipeline allocates four ports to connect to the two computers and the rest 12 ports to connect to the tester, respectively. This means that the router prototype can provide 1.2 Tbps forwarding rate at maximum. The two computers prepare cache spaces of 10^8 Kbytes data packets in total, which corresponds to 100 Gbytes DRAM space.

In the rest of this subsection, we describe how to implement caching functions on the switch and the computers.

b: IMPLEMENTATION ON SWITCH

We use the P4_16 language [23] and its Tofino extension [24] for the implementation on the switch. According to the cache architecture design in Sections III-A and III-B, the switch implements the cache admission and lazy response algorithms as described below.

The cache admission algorithm uses the insertion table to hold binary decision variables each of which is used to decide whether a data packet is inserted or not. The table is implemented by an SRAM-based array (called a *register* in P4) and each entry corresponds to the data name of interest, and the hash of a data name is used as the index to the register instead of the data name.

A key to implement the lazy response algorithm is to replicate delayed data packets transferred from the computer to the multiple ports indicated by the corresponding PIT entry. We use the Tofino ASIC's hardware unit for packet replication [24] for this replication.

c: IMPLEMENTATION ON COMPUTER

We use the Data Plane Development Kit (DPDK) [25] to implement the CS, FIB and PIT as software on a computer. We fully implement the CS which consists of a packet store according to a lock-free data structure [26], the TinyLFU cache algorithm [15], the cache admission and the lazy response algorithms of Sections III-A and III-B. In contrast, we minimally implement the FIB and PIT so that interest packets are equally forwarded to the two computers and that data packets are replied to the ports from which the corresponding interest packets are received.

B. MODEL VALIDATION

We validate that the router implementation achieves similar performance to the analytical model. For the validation, we use the same conditions as in Section V-A, except for the input interest packet rate for the router implementation. The input interest packet rate is set to 146 million packet/s (MPPS), the value of which saturates 12 100 Gbps links of the router implementation with 1024 bytes data packets.

Figure 15(a) shows the throughput values of the router implementation and the analytical model for the proposal router and the naive router. For both routers, the router implementation achieves similar throughput to the analytical model. As a result, the proposal router and the naive router achieve throughput of 1.075 Tbps and 0.491 Tbps, respectively, and the proposal router achieves 2.1 times higher throughput than the naive router.

Next, we validate that the router implementation has similar cache hit ratio p , cache insertion ratio q and cache response ratio r to the analytical model. Figure 15(b) shows p , q and r values of the router implementation and those of the analytical model. We only focus on the values for the

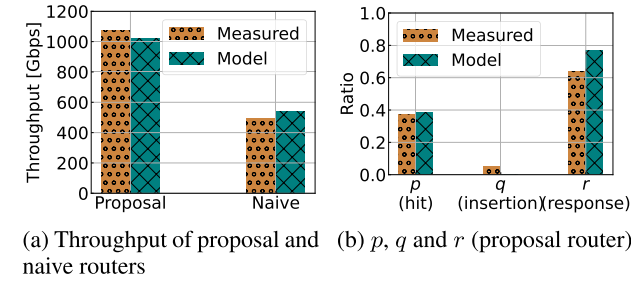


FIGURE 15. Model validation.

proposal router because the naive router always takes 1 as q and r values and has similar p value to the proposal router.

The rest of subsection summarizes the reasons that the three ratios in the router implementation are slightly different from those in the analytical model. First, cache insertion ratio q in the router implementation is higher than that in the analytical model because unnecessary cache insertions occur in the router implementation due to data name hash collisions. Precisely, the router implementation uses data name hashes in the insertion table (as described in Section VI-A.b), while the analytical model uses exact data names. When unpopular data packets collide with popular ones in the data name hashes, the unpopular ones may be erroneously inserted to the cache, which increases q values in the router implementation.

Second, cache hit ratio p in the router implementation is lower than that in the analytical model because of the cache insertion of unpopular data packets explained above. Specifically, the inserted unpopular data packets evict popular data packets, which will cause cache hits, from the cache.

Third, cache response ratio r in the router implementation is lower than that in the analytical model because the cache misses explained above occur for data packets with high cache hit ratio and high cache response ratio. The definition of r (i.e., equation (11)) implies that such the cache misses reduce r value because the numerator ($p_k r_k$, k is for the data packets with the cache misses) decreases.

C. STABILITY AGAINST TRAFFIC DYNAMICS

Finally, we experimentally evaluate the performance stability under popularity dynamics, which appear in realistic traffic traces [3], as the analytical model does not evaluate this aspect. The aim of this evaluation is to validate whether or not the forwarding rate degrades when popularities of data packets change. To generate a request sequence of this validation, we use the same popularity distribution (i.e., Zipf distribution with skewness 0.9), but periodically change popular data packets based on a popularity dynamics model [19]: We move 10^5 least popular data packets to the top of the popularity ranks and lower the ranks of other data packets. We run the experiment for 600 seconds, changing the popularity every 60 seconds.

Figure 16(b) shows the values of cache hit ratio p , cache insertion ratio q and cache response ratio r for the proposal router. Please note that we show the values after 200 seconds elapse because the first 200 seconds are used for the router's

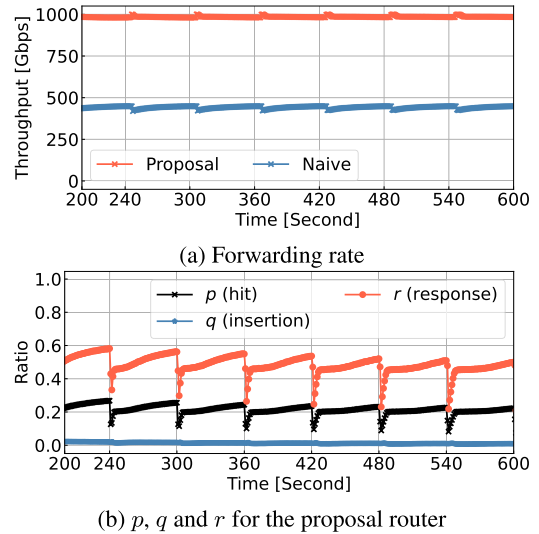


FIGURE 16. Performance stability under traffic dynamics (Popularity changes every 60 seconds).

cache to warm up. p value decreases every 60 seconds (e.g., at time 240) because popular data packets change every 60 seconds. Interest packets to the new popular data packets miss the cache from this time until they are cached. The cache misses during this period temporarily decrease cache hit ratio p .

However, we observe that the throughput of the proposal router is stable, that is, the throughput degradation is not remarkable against temporal cache hit rate drops. Figure 16 shows throughput values of the proposal router and the naive router. This result demonstrate the performance stability of the proposal router.

VII. RELATED WORK

This section compares this paper with other studies in terms of high-speed ICN router implementations, high-speed caching, cache admission and lazy responses.

There have been numerous studies that address high-speed ICN router implementations. Most studies focus on designing a compact forwarding data structure [27], [28] and a high-speed lookup algorithm for the data structure [26] to achieve high-speed name-based forwarding. A few studies propose data structures for the pending interest table, which keeps reverse paths of content requests, to store the reverse paths in the small (but fast) memory device [4], [29]. But, they do not carefully address high-speed caching [30], [31], [32]. That is, they use SSD devices as the caching storage, of which capacity is an order of magnitude larger than that of the DRAM device; however, the access speed is much less than terabit-class throughput.

Recent studies develop full-fledged ICN router implementations on hardware platforms of multi-core CPU servers and programmable switches. The router implementations on the former platforms achieve 100 Gbps-class throughput [5], [6], [33], [34], and those on the latter platform achieve terabit-class throughput [8], [35], [36]. However, they still only focus

on name-based forwarding. To the best of our knowledge, our study is first to addressing how to implement Tbps-class caching on the ICN router.

High-speed caching has been widely studied in research fields outside of ICN, especially KVS [19], [37], [38], [39], [40], [41], [42], [43]. Among them, some studies [19], [37], [38], [39], [40], [41] use a similar system, which consists of a switch and computers, to our system with the similar goal, terabit-class throughput. Their cache systems leverage computers as (large) backend cache memories, but the use of a programmable switch differs from our study. Some studies [19], [38], [39] use the programmable switch as a small (frontend) cache for the backend cache memories. Such an in-switch cache makes requests hit at the switch's cache, which achieves the similar goal to our study, i.e., reducing traffic between the switch and computers. However, this design has two limitations. First, the amount of traffic reduction in the in-switch cache is smaller than that in our study, because the small cache memory (tens of Mbytes) at the switch limits the cache hit ratio. For example, the reduction rate in the in-switch cache is at most 30%, which is much smaller than that in our design (70% reduction). Second, the in-switch cache design only supports the short payload size (up to 128-bytes), which is much smaller than the ICN's payload size (1024 bytes), to cache, and thus this design is not applicable to ICN routers' caches. Hence, we do not employ the in-switch cache design for our cache system.

Other studies [37], [40], [41] in the KVS field use a switch as a load balancer for backend cache memories on computers in conjunction with *selective replication*. The selective replication replicates a small set of highly popular key-value pairs, which account for most of requests, on all the computers. The switch load balancer distributes requests to these pairs across all the computers by keeping and referring to replicated keys at the switch's memory. This design does not address the traffic reduction between the switch and computers, which we address as a research issue. However, this design solves load imbalance across computers, which leads to performance degradation, while our study did not focus on. We can integrate this design with our cache system to address such a load imbalance issue by using hashes of popular content names in keeping popular keys. This design only consumes a few Kbytes of switch memory, which fits into the resource consumption in our implementation.

The concept of cache admission itself, which decides whether to cache incoming content objects or not, has been widely used by caching systems. Initially, cache admission is used to improve the cache hit ratio [15], [44], [45]. In contrast, other studies including our study shed light on traffic reduction in hierarchical cache systems. Some studies [46], [47], [48] aim at reducing traffic between a primary memory device (e.g., DRAM device) and a secondary storage device (e.g., SSD device). These studies use cache admission to reduce write traffic to the SSD device, as the SSD device has a limited number of writes in its lifespan. The motivation of our paper, cache system performance improvement, is different

from the studies, but the similar objective (i.e., traffic reduction) is achieved. Our paper's difference from the above studies is the implementation style that the cache admission is implemented on the switch. Thus, we carefully address implementation issues such as high throughput and efficient resource utilization on a constrained switch ASIC.

The objective of the lazy response algorithm is to aggregate multiple requests to the same object so that a single response is replied. Such aggregation techniques are studied in different systems from high-speed caching systems: ICN routers and video on-demand systems. First, in ICN, the similar technique called request aggregation [1], [2], [49] is used to reduce replies from a content server to a router. They work as a requester and a responder, respectively, similarly to a switch and a computer of our paper. In this technique, the router delays sending incoming requests to the content server for a round-trip time of $O(10)$ ms. A single content object is replied for multiple requests received by the router. The objective of this technique is the same as that of lazy response algorithm of our paper. A difference is which is delayed either a request or a response. Another difference is that our paper analytically evaluates the relation between delay values and packet forwarding rates to obtain a guideline of determining the appropriate value.

Second, the idea of adding such a response delay is considered in the context of video on-demand systems, a few decades ago [50], [51], [52] because the same video is likely to be requested for a short duration by multiple users. One of their motivations, traffic reduction, is similar to that of our paper; however, the factors that determine delay thresholds for aggregation are different from those in our paper. These studies analyze the relation between users' drop (unsubscribe) rates and delay thresholds because they are afraid that they may do so due to large delay incurred by the aggregation. In contrast, our paper focuses on another type of relation. That is, we analyze the relation of forwarding rate and delay thresholds with a more sophisticated model where hardware constraints and cache admission are considered. Besides, our paper focuses on high-speed implementation on hardware platforms.

VIII. CONCLUSION

This paper presents a terabit-class cache architecture of ICN routers by combining a programmable switch and commodity computers acting as a packet forwarder and large-capacity cache memory devices, respectively. First, we analytically show that the traffic for sending data packets to and from computers causes the performance bottleneck of a naive cache implementation. To solve this bottleneck, we design the two algorithms: the *cache admission algorithm* to reduce the traffic of inserting newly arrived data packets to the computers, and the *lazy response algorithm* to reduce the traffic for responding requested data packets from the computers. Second, we develop an analytical model of a cache system with the two algorithms to show that the two algorithms perform well under realistic request patterns

following the Zipf distribution. Finally, we implement an ICN router with the cache functionality using a 3.2 Tbps Tofino switch with two pipelines. We experimentally show 1.075 Tbps packet forwarding rate with a single pipeline, which is 2.1 times higher than the 0.491 Tbps packet forwarding rate in the naive implementation without the proposed algorithms. These efforts represent a promising step towards the deployment of ICN (and in-network caching) in terabit-scale networks, enabling traffic and response time reductions at Internet scale.

We summarize the future directions as follows: One direction is to extend our design to the multi-pipeline ASIC, to increase forwarding rate further. State management across multiple pipelines remains an issue because a packet arriving at one pipeline cannot access to the state (e.g., cache admission flag in our case) in another pipeline due to the hardware constraints. Another direction is to explore how to reduce interest packet traffic, which also becomes another bottleneck cause by small data packets. A candidate solution is hierarchical caching, where in-switch caches run in front of computers.

REFERENCES

- [1] V. Jacobson, D. K. Smetters, J. D. Thornton, M. F. Plass, N. H. Briggs, and R. L. Braynard, "Networking named content," in *Proc. ACM CoNEXT*, Dec. 2009, pp. 1–12.
- [2] L. Zhang, "Named data networking," *SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 4, pp. 66–73, 2014.
- [3] A. Sabnis and R. K. Sitaraman, "Tragen: A synthetic trace generator for realistic cache simulations," in *Proc. ACM IMC*, 2021, doi: [10.1145/3487552.3487845](https://doi.org/10.1145/3487552.3487845).
- [4] H. Yuan and P. Crowley, "Scalable pending interest table design: From principles to practice," in *Proc. IEEE Conf. Comput. Commun.*, Apr. 2014, pp. 2049–2057, doi: [10.1109/INFOCOM.2014.6848146](https://doi.org/10.1109/INFOCOM.2014.6848146).
- [5] J. Takemasa, Y. Koizumi, and T. Hasegawa, "Data prefetch for fast NDN software routers based on hash table-based forwarding tables," *Comput. Netw.*, vol. 173, May 2020, Art. no. 107188, doi: [10.1016/j.comnet.2020.107188](https://doi.org/10.1016/j.comnet.2020.107188).
- [6] J. Shi, D. Pesavento, and L. Benmohamed, "NDN-DPDK: NDN forwarding at 100 Gbps on commodity hardware," in *Proc. ACM ICN*, 2020, pp. 30–40.
- [7] (2021). *Intel*. [Online]. Available: <https://www.intel.com/content/www/us/en/products/network-io/programmable-ethernet-switch/tofino-2-series/tofino-2.html>
- [8] J. Takemasa, Y. Koizumi, and T. Hasegawa, "Vision: Toward 10 Tbps NDN forwarding with billion prefixes by programmable switches," in *Proc. ACM ICN*, 2021, pp. 13–19.
- [9] J. Takemasa, Y. Koizumi, and T. Hasegawa, "Terabytes and Terabits/s packet caching in ICN routers using programmable switches," in *Proc. IEEE 11th Int. Conf. Cloud Netw. (CloudNet)*, Nov. 2022, pp. 67–72.
- [10] N. Mohan, L. Corneo, A. Zavodovski, S. Bayhan, W. Wong, and J. Kangasharju, "Pruning edge research with latency shears," in *Proc. ACM HotNets*, Nov. 2020, pp. 1–26, doi: [10.1145/3422604.3425943](https://doi.org/10.1145/3422604.3425943).
- [11] (2021). *A+ Server 1024us-trt*. [Online]. Available: <https://www.supermicro.com/en/Aplus/system/1U/1024/AS-1024US-TRT.cfm>
- [12] (2021). *AMD EPYC™ 7702*. [Online]. Available: <https://www.amd.com/en/products/cpu/amd-epyc-7702>
- [13] (2021). *100GbE Intel*. [Online]. Available: <https://ark.intel.com/content/www/us/en/ark/products/series/184846/100gb-e-intel-ethernet-network-adaptor-e810.html>
- [14] J. Takemasa, A. Tagami, Y. Koizumi, and T. Hasegawa, "Load balancing for stateful forwarding by mitigating heavy hitters: A case for multi-threaded NDN software routers," *IEEE Access*, vol. 8, pp. 155071–155085, 2020, doi: [10.1109/ACCESS.2020.3018555](https://doi.org/10.1109/ACCESS.2020.3018555).
- [15] G. Einziger, R. Friedman, and B. Manes, "TinyLFU: A highly efficient cache admission policy," *ACM Trans. Storage*, vol. 13, no. 4, pp. 2–20, 2017.
- [16] L. Saino, I. Psaras, and G. Pavlou, "Understanding sharded caching systems," in *Proc. IEEE INFOCOM 35th Annu. IEEE Int. Conf. Comput. Commun.*, Apr. 2016, pp. 1–9, doi: [10.1109/INFOCOM.2016.7524442](https://doi.org/10.1109/INFOCOM.2016.7524442).
- [17] V. Martina, M. Garetto, and E. Leonardi, "A unified approach to the performance analysis of caching systems," in *Proc. IEEE Conf. Comput. Commun.*, Apr. 2014, pp. 2040–2048, doi: [10.1109/INFOCOM.2014.6848145](https://doi.org/10.1109/INFOCOM.2014.6848145).
- [18] P. Jurkiewicz, G. Rzym, and P. Borylo, "Flow length and size distributions in campus Internet traffic," *Comput. Commun.*, vol. 167, pp. 15–30, Feb. 2021, doi: [10.1016/j.comcom.2020.12.016](https://doi.org/10.1016/j.comcom.2020.12.016).
- [19] X. Jin, X. Li, H. Zhang, R. Soule, J. Lee, N. Foster, C. Kim, and I. Stoica, "NetCache: Balancing key-value stores with fast in-network caching," in *Proc. 26th Symp. Operating Syst. Princ.*, Oct. 2017, pp. 121–136, doi: [10.1145/3132747.3132764](https://doi.org/10.1145/3132747.3132764).
- [20] J. Yang, Y. Yue, and K. V. Rashmi, "A large-scale analysis of hundreds of in-memory key-value cache clusters at Twitter," *ACM Trans. Storage*, vol. 17, no. 3, pp. 1–35, Aug. 2021, doi: [10.1145/3468521](https://doi.org/10.1145/3468521).
- [21] (2021). *Wedge 100BF-32X*. [Online]. Available: <https://www.edgecore.com/productsInfo.php?cls=1&cls2=5&cls3=181&id=335>
- [22] (2021). *Inte*. [Online]. Available: <https://ark.intel.com/products/91317>
- [23] (2021). *P416 Language Specification*. [Online]. Available: <https://p4.org/p4-spec/docs/P4-16-v1.2.1.pdf>
- [24] (2021). *Open Tofino*. [Online]. Available: <https://github.com/barefootnetworks/Open-Tofino>
- [25] (2021). *Data Plane Development Kit (DPDK)*. [Online]. Available: <https://www.dpdk.org/>
- [26] W. So, A. Narayanan, and D. Oran, "Named data networking on a router: Fast and DoS-resistant forwarding with hash tables," in *Proc. Arch. Netw. Commun. Syst.*, Oct. 2013, pp. 215–225, doi: [10.1109/ANCS.2013.6665203](https://doi.org/10.1109/ANCS.2013.6665203).
- [27] T. Song, H. Yuan, P. Crowley, and B. Zhang, "Scalable name-based packet forwarding: From millions to billions," in *Proc. 2nd ACM Conf. Inf-Centric Netw.*, Sep. 2015, pp. 19–28, doi: [10.1145/2810156.2810166](https://doi.org/10.1145/2810156.2810166).
- [28] Z. Li, J. Liu, L. Yan, B. Zhang, P. Luo, and K. Liu, "Smart name lookup for NDN forwarding plane via neural networks," *IEEE/ACM Trans. Netw.*, vol. 30, no. 2, pp. 529–541, Apr. 2022, doi: [10.1109/TNET.2021.3119769](https://doi.org/10.1109/TNET.2021.3119769).
- [29] S. Jang, H. Byun, and H. Lim, "Dynamically allocated Bloom filter-based PIT architectures," *IEEE Access*, vol. 10, pp. 28165–28179, 2022, doi: [10.1109/ACCESS.2022.3158368](https://doi.org/10.1109/ACCESS.2022.3158368).
- [30] R. B. Mansilha, L. Saino, M. P. Barcellos, M. Gallo, E. Leonardi, D. Perino, and D. Rossi, "Hierarchical content stores in high-speed ICN routers: Emulation and prototype implementation," in *Proc. 2nd ACM Conf. Inf-Centric Netw.*, Sep. 2015, pp. 59–68, doi: [10.1145/2810156.2810159](https://doi.org/10.1145/2810156.2810159).
- [31] W. So, T. Chung, A. Narayanan, H. Yuan, D. Oran, and M. Stapp, "Toward terabyte-scale caching with SSD in a named data networking router," in *Proc. ACM/IEEE ANCS*, 2014, pp. 241–242, doi: [10.1145/2658260.2661767](https://doi.org/10.1145/2658260.2661767).
- [32] T. Pan, X. Lin, E. Song, C. Xu, J. Zhang, H. Li, J. Lv, T. Huang, B. Liu, and B. Zhang, "NB-cache: Non-blocking in-network caching for high-performance content routers," *IEEE/ACM Trans. Netw.*, vol. 29, no. 5, pp. 1976–1989, Oct. 2021, doi: [10.1109/TNET.2021.3083599](https://doi.org/10.1109/TNET.2021.3083599).
- [33] D. Kirchner, R. Ferdous, R. L. Cigno, L. Maccari, M. Gallo, D. Perino, and L. Saino, "Augustus: A CCN router for programmable networks," in *Proc. 3rd ACM Conf. Inf-Centric Netw.*, Sep. 2016, pp. 31–39, doi: [10.1145/2984356.2984363](https://doi.org/10.1145/2984356.2984363).
- [34] E. Newberry, X. Ma, and L. Zhang, "YanFD: Yet another named data networking forwarding daemon," in *Proc. 8th ACM Conf. Inf-Centric Netw.*, Sep. 2021, pp. 30–41.
- [35] R. Miguel, S. Signorello, and F. M. V. Ramos, "Named data networking with programmable switches," in *Proc. IEEE 26th Int. Conf. Netw. Protocols (ICNP)*, Sep. 2018, pp. 400–405, doi: [10.1109/ICNP.2018.00055](https://doi.org/10.1109/ICNP.2018.00055).
- [36] X. Long, K. Huang, R. Yang, Q. Dai, and Z. Li, "Pegasus: A high-speed NDN router with programmable switches and server clusters," in *Proc. 10th ACM Conf. Information-Centric Netw.*, Oct. 2023, pp. 12–18, doi: [10.1145/3623565.3623713](https://doi.org/10.1145/3623565.3623713).
- [37] J. Li, J. Nelson, E. Michael, X. Jin, and D. R. K. Ports, "Pegasus: Tolerating skewed workloads in distributed storage with in-network coherence directories," in *Proc. USENIX OSDI*, 2020, pp. 387–406.

- [38] S. Sheng, H. Puyang, Q. Huang, L. Tang, and P. P. C. Lee, "FarReach: Write-back caching in programmable switches," in *Proc. USENIX ATC*, 2023, pp. 571–584.
- [39] R. Friedman, O. Goaz, and D. Hovav, "Limited associativity caching in the data plane," 2022, *arXiv:2203.04803*.
- [40] X. Zheng, Z. Zhou, J. Lv, D. Zhang, and C. Wu, "KVLB: An in-network key-value load balancer using multi-valued hash," in *Proc. IEEE Global Commun. Conf.*, Dec. 2022, pp. 3766–3771, doi: [10.1109/GLOBE-COM48099.2022.10001265](https://doi.org/10.1109/GLOBE-COM48099.2022.10001265).
- [41] H. Eldakiky and D. H. Du, "Scaling up the performance of distributed key-value stores with in-switch coordination," in *Proc. 29th Int. Symp. Model., Anal., Simul. Comput. Telecommun. Syst. (MASCOTS)*, Nov. 2021, pp. 1–8, doi: [10.1109/MASCOTS53633.2021.9614283](https://doi.org/10.1109/MASCOTS53633.2021.9614283).
- [42] H. Lim, D. Han, D. G. Andersen, and M. Kaminsky, "MICA: A holistic approach to fast in-memory key-value storage," in *Proc. 11th USENIX Symp. Neww. Syst. Des. Implement. (NSDI)*, 2014, pp. 429–444.
- [43] I. Marinos, R. N. M. Watson, M. Handley, and R. R. Stewart, "DiskCryptnet: Rethinking the stack for high-performance video streaming," in *Proc. Conf. ACM Special Interest Group Data Commun.*, Aug. 2017, pp. 211–224, doi: [10.1145/3098822.3098844](https://doi.org/10.1145/3098822.3098844).
- [44] V. Kirilin, A. Sundarajan, S. Gorinsky, and R. K. Sitaraman, "RL-cache: Learning-based cache admission for content delivery," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 10, pp. 2372–2385, Oct. 2020, doi: [10.1109/JSAC.2020.3000415](https://doi.org/10.1109/JSAC.2020.3000415).
- [45] K. Liu, K. Wu, H. Wang, K. Zhou, P. Wang, J. Zhang, and C. Li, "SLAP: Segmented Reuse-Time-Label based admission policy for content delivery network caching," *ACM Trans. Archit. Code Optim.*, vol. 21, no. 2, pp. 1–24, Jun. 2024, doi: [10.1145/3646550](https://doi.org/10.1145/3646550).
- [46] T.-W. Yang, S. Pollen, M. Uysal, A. Merchant, and H. Wolfmeister, "CacheSack: Admission optimization for Google datacenter flash caches," in *Proc. USENIX ATC*, 2022, pp. 1021–1036.
- [47] J. Yang, N. Plasson, G. Gillis, N. Talagala, S. Sundararaman, and R. Wood, "HEC: Improving endurance of high performance flash-based cache devices," in *Proc. ACM SYSTOR*, 2013, pp. 1–11, doi: [10.1145/2485732.2485743](https://doi.org/10.1145/2485732.2485743).
- [48] D. L.-K. Wong, H. Wu, C. Molder, S. Gunasekar, J. Lu, S. Khandkar, A. Sharma, D. S. Berger, N. Beckmann, and G. R. Ganger, "Baleen: ML admission & prefetching for flash caches," in *Proc. USENIX FAST*, 2024, pp. 347–371.
- [49] P. Bakhtou and S. Khorsandi, "Joint request aggregation and content caching at the edge via named data networking," in *Proc. 29th Iranian Conf. Electr. Eng. (ICEE)*, May 2021, pp. 601–606, doi: [10.1109/ICEE52715.2021.9544137](https://doi.org/10.1109/ICEE52715.2021.9544137).
- [50] A. Dan, D. Sitaram, and P. Shahabuddin, "Scheduling policies for an on-demand video server with batching," in *Proc. ACM MULTIMEDIA*, 1994, pp. 15–23, doi: [10.1145/192593.192614](https://doi.org/10.1145/192593.192614).
- [51] C. C. Aggarwal, J. L. Wolf, and P. S. Yu, "On optimal batching policies for video-on-demand storage servers," in *Proc. 3rd IEEE Int. Conf. Multimedia Comput. Syst.*, vol. 3, May 1996, pp. 253–258, doi: [10.1109/MMCS.1996.534983](https://doi.org/10.1109/MMCS.1996.534983).
- [52] F. A. Pinto, E. M. G. Fernández, and C. M. Pedrosa, "Exploiting user preferences to reduce bandwidth requirements for VoD services with client caching," *IET Commun.*, vol. 15, no. 6, pp. 763–770, Apr. 2021, doi: [10.1049/cmu2.12118](https://doi.org/10.1049/cmu2.12118).



JUNJI TAKEMASA (Member, IEEE) received the master's and Ph.D. degrees in information science from Osaka University, Japan, in 2016 and 2019, respectively. After receiving the Ph.D. degree, he was a Research Engineer with KDDI Research Inc., for one and half years and moved to Osaka University. He is currently an Assistant Professor with the Graduate School of Information and Science, Osaka University. His research interests include programmable networks, future internet, and anonymous communication. He is a member of IEICE and IPSJ.



YUKI KOIZUMI (Member, IEEE) received the Ph.D. degree from Osaka University, in 2009. He is currently an Associate Professor with the Graduate School of Information Science and Technology, Osaka University, Japan. His research interests include privacy and programmable data plane.



TORU HASEGAWA (Member, IEEE) received the B.E., M.E., and Dr. (Informatics) degrees in information engineering from Kyoto University, Japan, in 1982, 1984, and 2000, respectively. After receiving the master's degree, he was a Research Engineer with KDDI Research and Development Laboratories (former KDD R&D Labs.), for 29 years, and then he was a Professor with the Graduate School of Information and Science, Osaka University, for 11 years. He is currently a specially-appointed Professor with Shimane University. He has published more than 100 papers in peer-reviewed journals and international conference proceedings, including *MobiCom*, *ICNP*, *IEEE/ACM TRANSACTIONS ON NETWORKING*, and *Computer Networks*. His current research interests include future internet, information-centric networking, and mobile computing. He is a fellow of IPSJ and IEICE. He received the Meritorious Award on Radio of ARIB, in 2003, the Best Tutorial Paper Award from IEICE, in 2014, and The Best Paper Award from IEICE, in 2015. He has served on the program or organization committees for several networking conferences, such as *ICNP*, *P2P*, *ICN*, *CloudNet*, *ICC*, and *GLOBECOM*, and as the *TPC Co-Chair* for *Testcom/Fates 2008*, *ICNP 2010*, *P2P 2011*, and *Global Internet Symposium 2014*.

• • •